Latest update: 10/28, 16:00

Assignment requirements: For security reasons, all submitted assignments will be executed inside a VM on the server, where they will be automatically graded, scored, and performance-tested. The environment is as follows:

1. Ubuntu 24.04
2. Compiler: GCC, standard C++17 recommended
3. 8GB RAM
4. Execution time limit per test case: 60 seconds (including file I/O, Stage 1, and Stage 2)
5. Build time limit: 60 seconds
6. No network access inside the VM

But don't worry, if you write your code normally and only use the C++ standard library, the OS, compiler, and RAM should not be an issue.

Build and execution procedure: The server will build and run your program as follows:

```
cd M123123123
cmake -B build
cmake --build build
cd build
# test files are copied into build/
./dp input001 output001
```

The compiled program must be located at build/dp (not build/src/dp or elsewhere). All input files are placed in build/, and all output files must be written to build/, i.e., the current working directory.

Required directory structure: After building and running, the project should ideally look like this:

```
M123123123/
  .git/                # optional, if you use git
  build/
    dp                 # compiled program
    input001.aux       # test data (copied into build/)
    input001.nodes
    input001.nets
    input001.pl
```

```
    input001.scl
    input001.wts
    output001.aux      # output result in build directory
    output001.nodes
    output001.nets
    output001.pl
    output001_legal.pl
    output001.scl
    output001.wts
  src/                 # source code
  CMakeLists.txt
  README.md
```

At submission, the project should ideally look like this:

```
M123123123/
  .git/                # optional, if you use git
  src/                 # source code
  CMakeLists.txt
  README.md
```

Note: This directory structure may look a bit weird, but it is designed to be consistent with Project03.pdf. Therefore, there are no separate input/ and output/ directories.

Submission notes:

1. Do not upload the build/ directory. The server will build the program, copy the test cases into build/, and run it on its own.
2. The project directory must be named after your student ID and compressed as `StudentID.zip`. ex: M11515013.zip
3. Each test case is a set of GSRC Bookshelf files sharing one base name: input001.aux, input001.nodes, input001.nets, input001.pl, input001.scl, input001.wts. Base names use three digits: input001, input002, ... Output base names must match: output001, output002, ... (A name like input1 is invalid; it must use three digits.)
4. For output base name outputXXX, the program must produce outputXXX_legal.pl (Stage 1) and outputXXX.pl (Stage 2), plus outputXXX.aux, .nodes, .nets, .scl, .wts.
5. In addition to the program, you must submit a report in PDF format. .md and .docx files are not accepted. Name the report after your student ID and upload it

as `StudentID.pdf` together with `StudentID.zip`. Do not put the report inside `StudentID.zip`.

6. The compiled program must be named `dp`. You can specify the name in CMakeLists.txt.
7. The first two arguments of the dp program must be the input base name and the output base name (without file extensions), respectively. If not enough arguments are provided, the program must report an error message.

HINT: The test cases will be derived from the benchmarks at

https://vlsicad.eecs.umich.edu/BK/Slots/cache/er.cs.ucla.edu/benchmarks/ibm-place2/

Most of these benchmarks contain macros. As stated in Project03.pdf, we simplify the problem, so the test cases will be further modified. If you have any questions about test case size or similar details, please download the benchmarks and take a look. They will be quite close to the actual test cases. However, you should set a time limit in your algorithm.